

# **Deep Learning Systems: Landmark Image Classification with a From-Scratch CNN**

Lo Kai Cheung, Stanley

Udacity Institute of AI & Technology

MSc in Artificial Intelligence — Capstone Project 4: Deep Learning Systems

[23 September 2026]

# **Deep Learning Systems: Landmark Image Classification with a From-Scratch CNN**

## **Report Overview**

This report documents the design, training, and evaluation of a convolutional neural network (CNN) for landmark image classification. The model classifies photographs into one of 50 landmark categories using a curated ~6,000-image subset of Google's Landmarks Dataset v2. A baseline CNN trained from scratch was compared against an experimental configuration that changed only the optimizer, isolating the effect of that single design decision on training dynamics and final performance.

## **Dataset and Task Description**

The dataset is a 50-class, approximately 6,000-image subset of Google's Landmarks Dataset v2 (Weyand et al., 2020), a large-scale collection of real-world landmark photographs originally assembled from geotagged Wikimedia Commons images. This particular subset was curated and distributed for coursework use via Udacity's Deep Learning Nanodegree. The images are pre-split into 4,996 training/validation images and 1,250 held-out test images, roughly 100 to 140 images per class, with class labels given by folder names that combine a numeric landmark identifier with a human-readable name (e.g., "07.Stonehenge").

This dataset directly supports the target task, image classification, because it provides a modest but sufficient number of labeled, real-world photographs per class, at a scale appropriate for training a CNN from scratch without requiring distributed infrastructure. Each image was resized to 256 pixels on the short side and cropped to a uniform 224 by 224 input, with per-channel normalization statistics computed directly from the training data rather than reused from a generic benchmark such as ImageNet, since the landmark-photo distribution is narrower and visually distinct from general-purpose image corpora.

## Model Architecture and Design Decisions

The baseline model is a CNN built entirely from scratch, with no transfer learning or pretrained weights. It consists of five sequential convolutional blocks, each composed of a 3x3 convolution, batch normalization, a ReLU activation, and 2x2 max pooling. Channel depth increases progressively from 32 to 512 across the five blocks, while spatial resolution is halved at each stage, from 224x224 down to 7x7. The resulting feature map is flattened and passed through a fully connected classifier head (25,088 to 1,024 units, ReLU, dropout, then 1,024 to 50 output units).

Several design choices shaped this architecture. Batch normalization (Ioffe & Szegedy, 2015) was placed after every convolutional layer to stabilize training and tolerate a higher learning rate than an unnormalized network would allow. Max pooling after each block keeps the parameter count manageable while progressively enlarging the receptive field. Dropout (Srivastava et al., 2014) was applied immediately before the final linear layer, specifically targeting the largest and most overfitting-prone transition in the network, the 25,088-to-1,024 fully connected layer, given the comparatively small size of the training set relative to the number of classes. The model outputs raw logits rather than a softmax distribution, since the cross-entropy loss function used for training applies log-softmax internally.

## Experimental Comparison

The experimental configuration changed exactly one factor relative to the baseline: the optimizer, from stochastic gradient descent (SGD) with momentum to Adam (Kingma & Ba, 2015). Architecture, data pipeline, batch size, epoch count, dropout rate, and weight decay were held identical across both runs. This was considered a meaningful comparison because the optimizer directly governs how gradients translate into weight updates, and therefore how

quickly and reliably a network converges, independent of its capacity or the data it is trained on. SGD with momentum is simple and well understood but can be slow to converge and sensitive to learning rate choice; Adam introduces per-parameter adaptive learning rates that are often more forgiving and can converge faster in practice.

An important methodological finding emerged during this comparison. Initially, Adam was trained at the same learning rate as the SGD baseline (0.01). This caused the experimental model's training to diverge entirely: both training and validation loss collapsed to approximately 3.91, the loss value corresponding to uniform random guessing across 50 classes, and remained there for the full training run. This is a known and well-documented interaction: Adam's adaptive, per-parameter scaling means it typically requires a substantially smaller base learning rate than SGD to train stably. Once the experimental model's learning rate was corrected to 0.001, a conventional default for Adam, training proceeded normally. This adjustment was treated as a methodological correction rather than a change to the single-axis comparison, since holding the raw learning rate constant across two optimizers with fundamentally different update rules does not constitute a controlled comparison of the optimizers themselves.

### **Results and Interpretation**

Over 30 training epochs, the baseline (SGD) model reached a final training loss of 2.486, a validation loss of 2.368, and a held-out test accuracy of 41.9% (test loss 2.200). The experimental (Adam) model reached a final training loss of 3.395, a validation loss of 3.217, and a test accuracy of 21.3% (test loss 3.109). Read as a single snapshot, this is a substantial gap in the baseline's favor.

However, the loss curves reveal a more specific and more interesting pattern than "SGD outperformed Adam." The baseline's validation loss declined steadily and had largely leveled off

by the later epochs, consistent with a model approaching convergence within the available training budget. The experimental model's validation loss, by contrast, was still declining at a meaningful rate through epoch 30, following a single learning-rate reduction around epoch 5, with no indication of having plateaued. This suggests the experimental configuration was still mid-convergence when training stopped, and that the reported gap reflects a difference in convergence speed under a fixed epoch budget at least as much as it reflects an inherent difference in the two optimizers' ceiling performance on this task. This distinction is corroborated qualitatively: reviewing individual test predictions from both models shows the experimental model correctly classifying visually distinctive landmarks while more frequently confusing visually similar ones, a pattern consistent with an under-trained but functioning classifier rather than a broken one.

### **Limitations and Risks**

Several limitations affect how these results should be interpreted. First, each configuration was trained for a single run; no averaging across multiple random seeds was performed, so some of the observed gap between configurations reflects run-to-run variance rather than a stable property of either optimizer. This was partially investigated by re-seeding the experimental model's initialization and confirming that its stagnation-then-recovery pattern was reproducible rather than an artifact of an unlucky initial state, but a fully rigorous comparison would average results across several seeds for each configuration.

Second, the dataset itself is limited in scale: approximately 100 to 140 images per class is a modest sample size for training a CNN from scratch, which constrains how much any architecture in this family can learn without either substantially more data or pretrained features from transfer learning. Third, the training and validation splits are drawn from the same

underlying pool of images (via an index-based split rather than independently collected sets), and represent a narrow geographic and cultural sample of landmarks, since the source dataset draws primarily from crowd-sourced, geotagged photography that does not sample the world's landmarks uniformly. Fourth, evaluation relies solely on top-1 classification accuracy and cross-entropy loss; no per-class breakdown, confusion analysis, or calibration assessment of predicted probabilities was performed, which would be needed to characterize where and why each model fails.

### **Ethical and Responsible Use**

A specific and plausible concern with this system is geographic and cultural representation bias. Landmark datasets assembled from crowd-sourced, geotagged photography are known to systematically over-represent landmarks in wealthier, more heavily photographed regions and under-represent those in lower-income or less-touristed parts of the world (Shankar et al., 2017). A classifier trained on such data would likely perform noticeably worse on landmarks from underrepresented regions than the aggregate 41.9% baseline accuracy suggests, even though the reported metric gives no visibility into that disparity. If deployed in a real application, such as automatic photo tagging or location suggestion, this could translate into a system that works well for some users' travel photos and poorly for others', in a pattern correlated with geography and, indirectly, with socioeconomic factors. Any deployment of this kind of model should include a per-region or per-class accuracy breakdown rather than relying on a single aggregate accuracy figure, and should be transparent to users about which landmarks the system is and is not reliable for.

### **Future Improvements**

Several concrete next steps could improve this system given additional time or resources. Training the experimental (Adam) configuration for additional epochs, given that its validation loss had not plateaued by epoch 30, would clarify whether it can close or exceed the gap with the SGD baseline rather than leaving that comparison incomplete. Averaging results across multiple random seeds for each configuration would separate genuine optimizer effects from initialization variance. Architecturally, replacing the large flatten-and-fully-connect classifier head with global average pooling would substantially reduce the parameter count and likely improve generalization given the dataset's modest size, and adding residual connections (He et al., 2016) could allow a deeper network to train more easily. Most substantially, initializing from ImageNet-pretrained weights rather than training from scratch would very likely outperform any amount of hyperparameter tuning on a dataset this size, since pretrained features from a much larger and more diverse image corpus would give the model a substantially stronger starting point than random initialization. Finally, a per-class and per-region accuracy breakdown, as noted above, would be a necessary next step before any real-world deployment.

## References

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 770–778. <https://doi.org/10.1109/CVPR.2016.90>
- Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of the 32nd International Conference on Machine Learning*, 448–456.
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/1412.6980>
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25, 1097–1105.
- Ruder, S. (2016). An overview of gradient descent optimization algorithms. *arXiv*. <https://arxiv.org/abs/1609.04747>
- Shankar, S., Halpern, Y., Breck, E., Atwood, J., Wilson, J., & Sculley, D. (2017). No classification without representation: Assessing geodiversity issues in open data sets for the developing world. *NeurIPS Workshop on Machine Learning for the Developing World*. <https://arxiv.org/abs/1711.08536>
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1), 1929–1958.

Weyand, T., Araujo, A., Cao, B., & Sim, J. (2020). Google Landmarks Dataset v2: A large-scale benchmark for instance-level recognition and retrieval. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2575–2584.  
<https://doi.org/10.1109/CVPR42600.2020.00265>